Securing Institutional Digital Asset Custody

Private Keys, Distributed Trust, and the Uncomfortable Arrival of Quantum Computing

Dimitris Konstantinou

Publication date 21 October 2026

GitHub github.com/demis1997

Research repository github.com/demis1997/quorum-custody

Transaction authority, distributed state, recovery, and cryptographic migration

Abstract

Institutional digital asset custody distributes cryptographic signing, transaction approval, operational state, and recovery across components whose security assumptions may not compose. This paper asks what remains enforceable when those trust domains fail independently and how the resulting architecture should evolve under a quantum threat to ECDSA. A revision pinned analysis of Quorum Custody is combined with 26 repeated unit tests, an executable divergent ledger model, unmodified recovery function experiments, and a local Ethereum transaction tested after approval expiry. The application broadcast validator rejected expired authorization while Anvil accepted the persisted ordinary transaction directly. Recovery rejected stale state against a retained checkpoint but accepted joint rollback of state and checkpoint under an explicitly privileged attacker. These findings illustrate known authority and state boundaries; they are not new cryptographic attacks or full MPC integration results. A local JavaScript verification benchmark measures selected ML-DSA, SLH-DSA, and hybrid configurations without inferring EVM gas or production performance. The paper contributes an evidence classified custody threat matrix and a staged migration proposal that covers spending, upgrades, recovery, and cryptographic agility. Native failure experiments and independent domain recovery remain unexecuted. The analysis supports conditional security claims tied to specific enforcement boundaries rather than a general claim of production readiness.

Keywords: institutional custody, threshold ECDSA, transaction authorization, distributed state, recovery, post quantum cryptography.

Contents

Abstract 2
1 Introduction 4
1.1 Research Contributions 4
2 Related Work 5
3 Research Methodology 5
4 Threat Model and Security Objectives 6
4.1 Institutional Compromise Matrix 7
5 Institutional Custody Architecture 8
6 MPC HSMs and Distributed Trust 10
7 Transaction Authorization and Policy Enforcement 11
7.1 The Signature Issuance Boundary 11
8 Distributed State Nonces and Recovery 12
9 Experimental Security Evaluation 14
9.1 Divergent Spending Histories 15
9.2 Expiry and Portable Transaction Authority 15
9.3 Recovery Rollback 15
9.4 Availability and Failure Handling 15
9.5 Test Evidence Quality 16
10 Quantum Threats to Ethereum Custody 16
11 Post Quantum Cryptographic Alternatives 17
11.1 Local Verification Microbenchmark 17
11.2 Institutional Suitability and Composition 18
12 Proposed Migration Architecture 19
12.1 Inventory and Account Preparation 20
12.2 Hybrid Enforcement and Asset Movement 20
12.3 Recovery Rotation and Cryptographic Agility 21
13 Discussion and Institutional Implications 21
14 Limitations and Future Research 22
15 Conclusion 22
References 23
Appendix A Reproducibility 25
Appendix B Experiment Protocols and Results 26

1 Introduction

A custody platform can produce a mathematically impeccable signature for a payment that nobody should have approved. The network will accept the signature according to its rules. It has no opinion about whether the finance team meant a different recipient, whether an administrator quietly changed the withdrawal policy, or whether the approval screen displayed the same transaction that reached the signer. Institutional custody therefore begins with a question about authority: which evidence entitles this particular transaction to move these particular assets?

The practical objective is to make unauthorized transfers difficult even when selected components fail or become hostile. A useful design must also preserve legitimate withdrawals when a participant disappears, maintain a coherent record after a crash, and recover without resurrecting old spending authority. Those requirements interact. An aggressive retry policy can improve availability while creating opportunities for duplicate authorization. A convenient recovery bundle can erase the separation that threshold signing was supposed to provide.

This paper develops three connected propositions. First, a signing threshold has operational meaning only when its participants occupy sufficiently independent trust domains. Second, transaction authorization requires agreement about exact bytes and about relevant state, including policy versions and consumed budgets. Third, an Ethereum ECDSA signature remains portable authority once issued, and its underlying public key remains exposed to the discrete logarithm threat posed by a sufficiently capable quantum computer. Neither a second approval screen nor an additional signing share changes that mathematical dependency.

Quorum Custody makes these propositions concrete. It is a local ETH transfer prototype using a 2-of-3 multiparty computation (MPC) threshold wallet and the Coinbase cb-mpc library. The inspected revision is 691e253ccdc2756afe215a23384eb72e2d4fde3c. Its restricted transaction surface is useful for analysis because the approval and lifecycle boundaries can be examined without pretending that arbitrary contract calls have the same semantics as an ETH transfer [1], [2]. The project is an engineering case study, not evidence that a production custodian has satisfied these propositions.

The central research question is whether institutional custody can preserve its authorization objectives when signing, approval, operational state, and recovery cross independently compromised trust domains, and how those objectives should evolve when ECDSA’s classical hardness assumption no longer holds. “Secure” is used here only with an explicit attacker capability and an explicit enforcement boundary. The paper does not answer the question with a universal yes.

1.1 Research Contributions

This study contributes an evidence classified analysis of the points where custody controls depend on one another, a reproducible set of application and local chain experiments, and a staged migration architecture that treats spending, upgrades, and recovery as related authorities. The divergent ledger example is a counterexample model, not a new MPC attack. The expiry experiment demonstrates a distinction between application validation and ordinary transaction validity. The recovery experiment tests the retained checkpoint assumption with unmodified repository functions. The migration architecture is a proposal requiring further implementation and review.

The distinction matters to an institutional reader. A cryptographic library can provide a protocol with stated assumptions. An application must preserve those assumptions while handling identity, failures, deployment, and people. The interesting work often starts precisely where the protocol’s proof ends. An architecture diagram with three signing boxes is encouraging. The administrator permissions behind those boxes tend to be more informative.

2 Related Work

Threshold ECDSA research establishes protocols for jointly producing an ordinary signature while protecting signing shares under stated assumptions. Gennaro and Goldfeder give a maliciously secure multiparty construction with dealerless setup [16]. That literature answers a cryptographic signing question. It does not establish that an application supplied the right transaction or maintained one coherent spending history. This paper does not identify the cb-mpc revision with that exact protocol solely because both implement threshold ECDSA.

Cryptographic module standards and key management guidance address complementary boundaries. FIPS 140-3 specifies requirements for modules, while NIST SP 800-57 Part 1 covers the protection and lifecycle of keying material and associated metadata [6], [19]. A custody service combines module access with application policy, identity, backup, and recovery. Module validation is not certification of that combination.

Distributed systems research separates agreement on state from the ability to authenticate a message. Practical Byzantine Fault Tolerance provides replicated service guarantees under its own fault and timing assumptions [17]. The Fischer, Lynch, and Paterson result limits deterministic consensus termination in a fully asynchronous model even with one stopping failure [18]. Neither result says that every custody system must use the same replication protocol. They show why a signing threshold cannot be treated as an unexplained substitute for consistent state and an explicit liveness model.

Shor identifies the algorithmic threat to discrete logarithm based cryptography [11]. NIST's ML-DSA, SLH-DSA, and ML-KEM standards distinguish transaction authentication candidates from key establishment [12], [13], [20]. Joseph et al. examine migration as an organizational and technical undertaking, including dependency discovery and transition planning [21]. Their migration argument motivates the authority inventory here; this paper's proposal applies that reasoning specifically to Ethereum custody.

Distributed post quantum signing is an active research topic. TOPCOAT develops a two party Dilithium related construction, and Tang et al. study functionally interchangeable lattice based threshold signatures [22], [23]. These are distinct constructions, not interchangeable labels for standardized ML-DSA or a drop in replacement for cb-mpc. Their results establish that distributed signing remains a research and engineering task after selecting a post quantum primitive. No such protocol is implemented or benchmarked here.

The case study's contribution lies in composition and evidence: tracing what each existing component actually enforces, testing selected boundaries, and identifying where additional authority would bypass an otherwise sound control. It is neither a new cryptographic primitive nor a proof of a general custody security theorem.

3 Research Methodology

The original baseline and the newly fetched public HEAD both resolve to 691e253ccdc2756afe215a23384eb72e2d4fde3c. No newer application revision is used. The original document, source JSON, builder, and test output are preserved in the research package. Additional experiments import functions from an isolated source checkout without modifying application behavior.

The investigation combines a revision pinned source review, execution of the repository's unit test files, and comparison with primary cryptographic standards and Ethereum specifications. Source review follows the approval binding in src/encoding.ts, transaction and policy validation in src/domain.ts, signer checks in src/signer.ts, and persistence and submission behavior in src/custody.ts and src/worker.ts. Recovery and its tests are reviewed as a separate security boundary. The native adapter is inspected for its use of the upstream access structure API; this is not an audit of cb-mpc's cryptographic implementation [1]–[5].

Three evidence categories remain separate throughout the paper. An observed source property describes a check or operation visible in the inspected revision. A locally executed result describes a command actually

run for this investigation. A historical result describes an outcome recorded by the repository at an earlier revision or execution. Passing a unit test is evidence for that test’s behavior in its execution environment; it is not independent confirmation of a deployed service, hardware isolation, or the complete adversary model.

The source contains several layers of verification history. The general verification record describes earlier native MPC, PostgreSQL, mTLS, Anvil, and browser exercises. The later engineering report describes additional checks at broadcast and explicitly lists new integration groups as pending. Consequently, earlier end to end success cannot be carried forward as proof of the revised submission path. The current analysis gives the later, more specific report priority when determining the scope of a claim [3], [5].

The executed suite uses synthetic keys and temporary state. It exercises application validation and encrypted recovery without starting the native MPC stack, PostgreSQL, or Anvil. Some broadcast fixtures deliberately use an ordinary complete Ethereum signing key to test the validation of signed bytes. Those fixtures do not establish threshold signing behavior. No operational custody latency, throughput, finality, hardware security, or quantum resource benchmark is derived from the unit tests.

Additional evidence consists of an executable quorum ledger model using real approval validation, recovery function experiments, a real Anvil transaction tested after application expiry, and a CPU verification microbenchmark. These are deliberately separate from native MPC integration. Docker was unavailable, PostgreSQL was not started, and the native adapter and patched OpenSSL were not built for this investigation. Full worker, quorum failure, and distributed recovery scenarios remain Not Executed.

Experiments ran on macOS ARM64 with an Apple M3 Max, Node 22.20.0, and the repository lockfile. The benchmark separately pins @noble/post-quantum 0.7.1 and uses @noble/curves 1.9.7. Complete commands, UTC event timestamps, numerical results, and declared scope are retained in appendices and machine readable logs. Anvil was bound only to loopback, funded with synthetic balance, and stopped by the harness after use.

4 Threat Model and Security Objectives

The protected assets include threshold shares, approval and policy signing keys, transport identities, approved transaction bytes, spending reservations, nonce history, and recovery checkpoints. The model considers an attacker who can steal an API bearer token, compromise one approver or signer, control the coordinator, modify application database state, or interfere with transport. These capabilities are considered separately unless a scenario explicitly combines them. A host administrator controlling the prototype’s single operating system account is a stronger adversary because that account owns the development signing and approval material [2].

The baseline signing assumption is that fewer than two threshold participants are compromised, the selected protocol is used correctly, its cryptographic assumptions hold, and honest participants retain their state and identities. For application authorization, enough approval keys must also remain uncompromised, approvers must review trustworthy transaction details, and signer policy state must be current enough for the claimed control. Application authentication alone does not satisfy those conditions. A stolen token and a stolen detached approval key confer different powers.

Safety means that an honest signer refuses a new signature request without sufficient valid evidence for the exact transaction under its accepted policy. This is narrower than saying that no unauthorized asset movement can ever occur. Two corrupted signers can bypass the application. Already issued signed transactions can be submitted outside its worker. Approved transactions may still reflect fraud or human error. Global spending guarantees require a consistent reservation history in addition to valid signatures.

Availability means that an eligible quorum can complete an authorized transfer when required dependencies remain available. A 2-of-3 design can tolerate one unavailable participant during signing, but the inspected distributed key generation path requires all three participants. A malicious signer may abort, and a

coordinator may suppress requests or exhaust bounded sessions. Availability is therefore conditional on workflow state, network behavior, remaining quorum, and resource controls. The same threshold does not describe every phase of the wallet's life.

Recovery safety requires restoration of both key material and the history that constrains its use. An old share with a fresh looking ledger can be more dangerous than an unavailable share. The quantum scenario introduces a different capability: an adversary able to solve the relevant elliptic curve discrete logarithm problem from a public key. It does not assume a date when that capability will exist. The analysis asks which custody controls would still bind such an attacker and which would be bypassed entirely.

There is also a mundane but decisive shared cause risk. If one deployment account can replace the signer binary everywhere, independent processes may execute identical malicious instructions. Moving those processes into separate directories does not remove that authority. Production separation must be expressed in actual administration, release, backup, and credential permissions, not inferred from process count.

4.1 Institutional Compromise Matrix

Table 1 distinguishes an attacker's capabilities from the authority a control is intended to constrain. "Function test" means an imported application function was exercised, not a live signer or whole deployment. "Model" means a declared counterexample model. All other rows are source based or analytical unless stated otherwise. The matrix is an assessment of this prototype and the proposed institutional design, not an assertion that every listed production control exists.

Table 1. Institutional custody compromise matrix

Scenario	Capabilities and boundary	Expected control	Residual risk	Evidence
Stolen API credentials	Call APIs within stolen role; authentication boundary	Detached approvals and signer validation	Denial of service; valid requests still possible	Source
Compromised browser	Mislead reviewer or steal imported approval key	Independent trusted display and approval device	False intent can receive valid approvals; hardened device absent	Source
One approval key	Issue one genuine approval	Distinct approvers and requester separation	Enough compromised keys authorize permitted transfers	Unit tests
Policy administrator	Broaden signed limits or recipients	Separate approval keys and pinned trust set	Policy protection weakened; manual rotation	Source
Coordinator	Choose work, deny service, retain signed bytes	Signer evidence checks and consistent reservation service	Portable signatures; stale policy and ledger divergence	A model; B function
Single MPC signer	Leak share, abort, falsify local state	Threshold protocol and independent domains	Denial of service; inconsistent history needs separate control	A model only
Threshold quorum	Generate signatures without application	Prevent common administration and protect shares	Direct asset movement possible	Analysis

Scenario	Capabilities and boundary	Expected control	Residual risk	Evidence
Malicious insider	Use granted role; collude with other authorities	Duty separation and independently reviewed intent	Collusion or misleading intent can satisfy formal policy	Unit tests only
Database manipulation	Alter reservations, state, approvals, audit	Detached signatures; external state integrity	Global budgets and audit depend on trusted state	Source
Signing host	Read keys, alter code or process memory	Independent hosts and external key protection	Shared prototype host defeats intended separation	Source
HSM API abuse	Invoke permitted signing without extracting key	Operation scoped authorization at module/service boundary	Arbitrary digest access may preserve signing abuse	Analysis; no HSM
Recovery infrastructure	Access backup password and comparison checkpoint	Independent rollback resistant checkpoints	Joint rollback accepted with declared privileges	C function
Stale state restore	Present old encrypted backup	Retained latest checkpoint	Whole checkpoint loss or rollback outside guarantee	C function
RPC provider	Lie about delivery, inclusion, balances	Independent observation and finality policy	Local Anvil trust; no public network failover	B Anvil only
Software supply chain	Replace signer or common dependencies	Pinned builds, release separation, review	One trusted update root can control all signers	Source
Quantum adversary	Derive classical key from known public key	Mandatory PQ authority and protected upgrades	Classical owner, guardian, delegation, network dependencies	Analysis

5 Institutional Custody Architecture

Figure 1 separates the desired institutional trust domains from the prototype’s actual deployment. The prototype runs its services and development key material under one host account. Mutual TLS, distinct processes, and separate directories exercise application interfaces; they do not create independent administrative control. Three deployment labels will not inconvenience an attacker who owns their shared update mechanism.

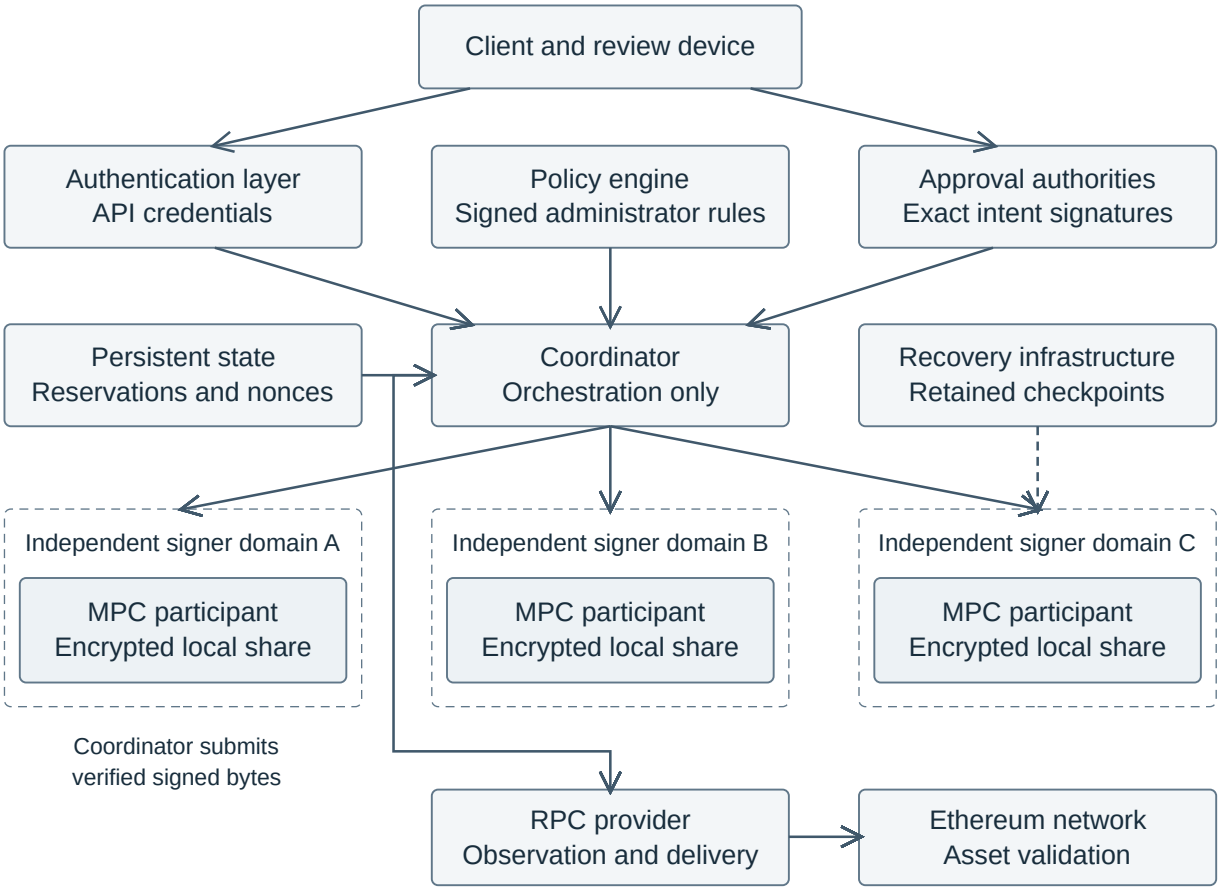


Figure 1. Institutional custody trust boundaries. Dashed signer domains and external protection describe proposed institutional administration. The prototype uses one host account; no HSM or KMS isolation is claimed. Representative paths are shown; each signer needs independently governed recovery.

Authentication should identify who may call an API. Policy should define the permitted asset intent. Approval authorities should attest to exact intent through a protected review path. Signers should verify the evidence they can independently check, then execute only a correctly scoped protocol. Persistent reservations and recovery checkpoints must preserve the history on which those decisions depend. RPC observation and network settlement add further trust assumptions after signing.

In an institutional deployment, signer domains should have distinct administration, wrapping authority, release approval, and recovery custody. A separate approval domain reduces the chance that coordinator compromise supplies both a malicious transaction and the signatures authorizing it. A shared disaster recovery operator or software update root can silently reconnect domains that look separate during normal operations. Figure 1 therefore shows a proposed administration boundary, not a production property demonstrated by Quorum Custody.

Key protection mechanisms must be named precisely. The case study uses encrypted local shares with wrapping keys on the same host, not HSM protected share execution. The desired architecture may introduce HSM or KMS protection, but its guarantees depend on the selected interface, configuration, and host attacker model. A diagram cannot promote a wrapping key into a certified signing boundary.

6 MPC HSMs and Distributed Trust

A hardware security module (HSM) aims to make sensitive key operations occur inside a protected module and to constrain access through that module's interfaces. FIPS 140-3 defines security requirements for cryptographic modules; a particular validation applies to a particular module and configuration [6]. The institutional question is how that boundary participates in authorization. If a compromised service is permitted to ask the module to sign any digest, preventing key extraction can still leave a fully functioning unauthorized signing service.

Threshold signing addresses a different problem: one participant should not possess enough signing authority to act alone. In a 2-of-3 wallet, any authorized pair can cooperate to produce a signature accepted under one public key. The participants use shares rather than sending a reconstructed private key to a coordinator. For a treasury withdrawal, one unavailable signer need not prevent the other two from proceeding. The security benefit depends on the protocol and on the inability of one adversary to control a sufficient set of participants.

The intuition is easier to understand before the algebra. Suppose the institution wants one Ethereum account but refuses to trust a single machine with complete signing power. Distributed key generation creates related secret state across participants and a common public key. During signing, the parties compute the signature jointly. The network receives a conventional signature and does not learn how the institution distributed its internal authority. An on chain multisignature account instead expresses multiple signatures or a threshold rule through contract execution. The two approaches expose different verification and governance boundaries.

For ECDSA, let G be a curve point of prime order n, x the secret scalar, and Q the public key. For message representative z and a valid nonzero per signature scalar k, the central relations are:

$$Q = xG, \quad R = kG, \quad r = R_x \bmod n, \quad s = k^{-1}(z + rx) \bmod n$$

The signer must handle zero values and all protocol checks required by its scheme. Verification uses the public key to test the signature relation. Threshold ECDSA jointly computes a compatible result while protecting the participants' secret state. The inverse and products in the expression for s explain why splitting a key is not equivalent to independently producing several ordinary signatures and averaging them [7].

Nonce handling deserves particular care. Reusing an ECDSA nonce across distinct messages can expose the secret scalar. RFC 6979 specifies a deterministic procedure for conventional signing, but it is not a recipe for inventing a distributed nonce protocol [8]. Threshold implementations need the randomness, consistency checks, proofs, and failure handling required by their own protocol. An application should use the reviewed public interface and avoid improvising recovery from cryptographic errors.

Quorum Custody's native adapter invokes cb-mpc's access structure key generation and signing functions and obtains a DER encoded signature for later verification. The reviewed architecture pins the upstream source and distinguishes the application's transport and authorization contributions from the library's cryptography [1], [4]. That separation is appropriate. It also means application tests cannot establish properties of the full native dependency graph.

HSMs and MPC can coexist, but compatibility must be demonstrated. A generic HSM signature interface may not support the arithmetic and protocol state a threshold scheme requires. A key management service (KMS) used only to unwrap a share protects a different boundary from hardware that executes the entire distributed protocol. The design should state where plaintext shares exist, who can invoke operations, and what a compromised host can observe. Buying a stronger box does not settle those questions on the purchase order.

7 Transaction Authorization and Policy Enforcement

The coordinator is useful precisely because it assembles work for other components. That convenience makes it a poor place to concentrate every security decision. If signers accept only a digest and an assertion that approval occurred, a compromised coordinator can choose the digest. Independent enforcement requires reviewable transaction bytes, authenticated approval evidence, an accepted policy, and a reliable mapping between the wallet and its public key.

In the inspected source, approvalText constructs canonical text with a domain tag and binds the wallet ID, transaction ID, requester, transaction digest, policy version, and expiration. authorize verifies the signed policy, parses the unsigned transaction, recomputes its digest, checks distinct approvers and role separation, and verifies their detached Ed25519 signatures. The signer additionally compares the supplied public key with its locally stored wallet public key before invoking the native signing path [1].

This creates a concrete distinction between credentials and authority. An API token may permit a request or an authenticated read. It does not supply the private approval keys needed to fabricate two valid detached signatures. Conversely, compromise of those approval keys can authorize transfers allowed by the accepted policy. Keeping the browser key in memory rather than returning it from the API removes one exposure path, but malicious browser code or a misleading approval interface remains consequential [2].

The supported transaction type is deliberately narrow: local chain 31337, EIP-1559 type 2, native ETH, empty calldata and access list, and a gas limit of 21,000. The exact type 2 signing payload binds its chain ID, nonce, fee parameters, destination, amount, calldata, and access list [9]. A recipient, fee, or nonce change therefore requires new approval evidence when the implementation verifies those bytes correctly. The restriction excludes token transfers and general contract execution, whose economic effects require substantially richer review.

The canonical serializer sorts object keys and preserves array order. It operates here within constrained application schemas. That is a useful implementation choice, not a general claim of interoperability with every JSON canonicalization standard or locale. Any future client implementation should reproduce the approved encoding byte for byte and test agreement with the signer. Human readable details should be decoded from those same bytes so that a polished screen cannot quietly describe a different request.

A malicious coordinator that alters the transaction cannot preserve honest approvals merely by recomputing the digest: the digest is inside the signed binding. It can still withhold service, reorder valid work, retain previously issued signatures, and exploit inconsistent state. Similarly, an insider with policy administration can broaden recipients or limits, but does not thereby acquire the separately pinned approval private keys. Meaningful separation requires both sets of permissions to remain independently controlled.

Signer independence is also limited by software diversity. Multiple signers calling the same parser can reject the same malformed transaction correctly, or accept the same semantic mistake together. Production review should examine the full path from display to decoding to signature. Repeating a decision three times is useful only after deciding which failures repetition can actually tolerate.

7.1 The Signature Issuance Boundary

Approval expiration is useful while an honest component is deciding whether to sign or submit. It is not embedded as a revocation rule in an ordinary Ethereum ECDSA transaction. Once a valid signed transaction exists, anyone holding its bytes can attempt to submit it while the network still considers it valid. An internal policy update cannot erase that capability from a compromised coordinator’s memory.

The current broadcast validation checks persisted signed bytes, recovered sender, current authorization, and the submission attempt limit. The worker performs this decision under policy and transaction locks, records authorization and the attempt, then releases the database transaction before RPC. Already observed

transactions reconcile first, even if the application's approval has expired. This ordering avoids claiming that a later administrative event reverses a network action already taken [1], [3].

Figure 2 traces the same intent from canonical bytes to network observation. Signer authorization is checked before native signing. Once a conventional signed transaction is returned, the bytes carry authority that no longer depends on the availability of the approval database.

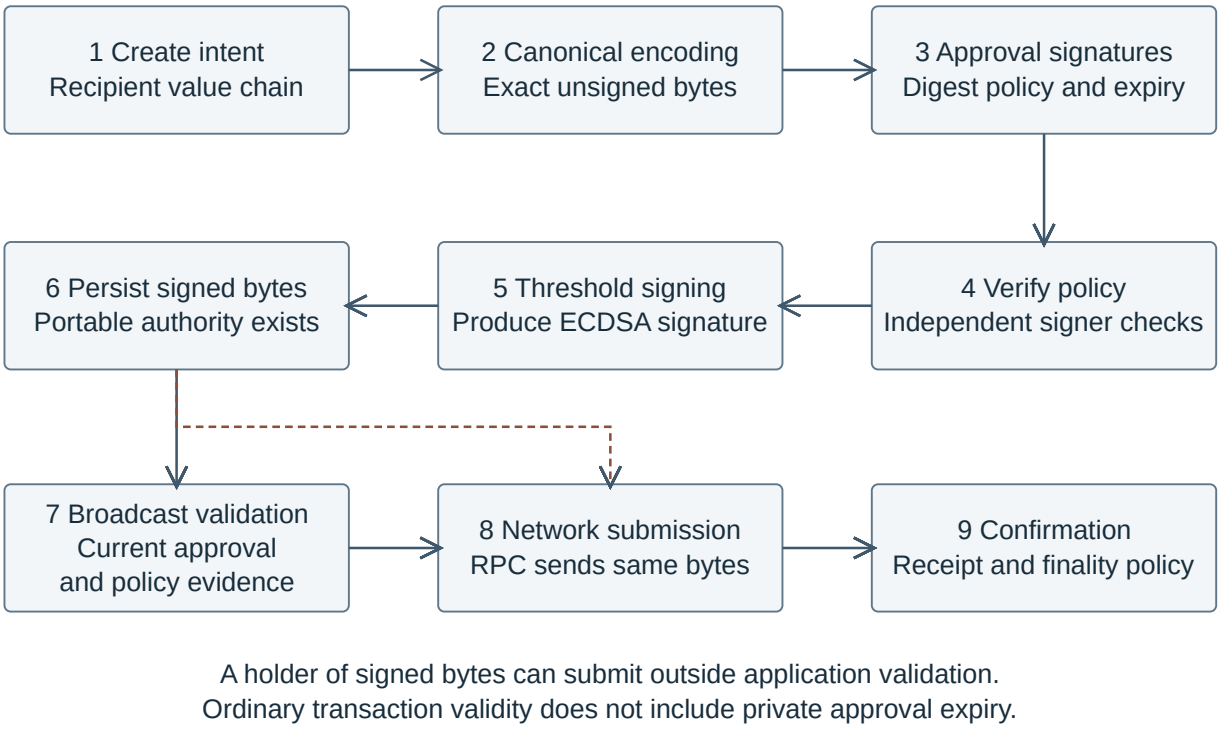


Figure 2. Transaction authorization lifecycle. The dashed path marks submission by another holder after ordinary signed bytes exist. Application expiry constrains honest components; network transaction validation has no knowledge of private approval expiry.

The remaining boundary is precise. A later freeze cannot recall an RPC already issued, and approval expiry can cross between the committed decision and network submission. A compromised coordinator can bypass its own software checks when it already holds portable signed bytes. Signer validation limits unauthorized new signatures; it cannot impose an off chain expiration on an existing ordinary transaction. Current policy update rules also refuse updates during active MPC, so the freeze mechanism is not an immediate emergency stop for every workflow state [3].

Code delegation requires a qualification to the original paper's broad EOA wording. EIP-7702 permits an EOA to delegate code, but its authorization mechanism still uses the account's ECDSA authority [26]. A recovered classical private key can therefore remain sufficient to change delegation or issue account transactions. Delegation should not be assumed to remove that root of authority. The proposed migration instead requires an account design whose spending and verifier replacement rules cannot be bypassed through a surviving classical owner path.

8 Distributed State Nonces and Recovery

Transaction validity is local to a transaction. A spending budget is a statement about a history. A payment can have valid signatures, an approved recipient, and a reasonable fee while still exceeding a wallet's

remaining authorized budget. This is where threshold signing and distributed systems meet: the signers must know enough consistent history to enforce the policy they believe they are enforcing.

Quorum Custody reserves a conservative upper bound for each transfer. With amount v, gas limit g, and maximum fee per gas f, the reservation is:

$$C = v + gf, B_{reserved} + C \leq B_{limit}$$

This bound avoids authorizing the amount while forgetting the maximum network cost. In the prototype, the aggregate is a lifetime wallet budget, not a rolling daily limit. PostgreSQL row locks serialize reservations and nonce allocation, while each signer retains a local ledger. Failed signing attempts keep conservative reservations. That choice favors safety but can strand capacity and requires an explicit operational resolution [1], [2].

A quorum intersection does not automatically create a globally consistent ledger. Consider three participants A, B, and C with the same overall spending limit. Each pair intersects another pair, but a malicious participant at the intersection may hide history. If A is corrupted, it can cooperate with B on one approved transfer and C on another. B and C can each observe a locally acceptable history while total spending exceeds the intended global budget. Valid approval signatures are assumed in this scenario; the attack concerns aggregate enforcement, not forgery. A trusted central reservation service prevents this particular divergence under its own assumptions. A Byzantine threat model requires an authenticated consistent reservation mechanism, or a carefully specified alternative [2].

Nonce checks constrain this scenario but do not solve every accounting problem. Different nonces may both be valid. Reusing one nonce can prevent both transactions from being included, but can also create ambiguous replacement authority and stalled later payments. An institution needs a defined rule for which intent owns each nonce, how fee replacement is approved, and how an abandoned reservation is reconciled. The prototype intentionally leaves several such operational flows unimplemented.

Persistent worker ownership addresses another class of failure. A worker may pause long enough for its lease to expire while another worker takes the task. Fencing checks reject writes from the stale owner. The reviewed worker persists the verified signed transaction and its hash before submission, then reconciles receipt and transaction lookup before sending the same bytes again. This supports recovery from an uncertain RPC response without asking MPC to create a different payment [1]. It does not guarantee exactly once network delivery or public chain finality.

The state called confirmed records observed inclusion, and may also carry a chain_reverted error. An operator must inspect the receipt outcome before claiming transfer success. A local Anvil receipt is further removed from an institutional finality policy with reorganization handling and independent observation. A status label is useful compression; it should not be asked to do the work of a settlement policy.

Recovery creates a similar distinction between useful checks and ultimate authority. The prototype restores encrypted signer state together with policy, ledgers, and replay records, and rejects an obsolete generation against a retained checkpoint. The checkpoint lives outside the signer directory but remains on the same host. This protects against selected stale restore mistakes; it cannot stop a host administrator who can rewrite both the restored state and its comparison checkpoint [2], [10].

A production recovery ceremony should preserve the independent control of each signer, protect passwords and wrapping keys separately, and rehearse loss of the database, identity material, and checkpoint service. The documented local workflow restores a signer directory while those other dependencies survive. Calling that whole platform disaster recovery would make the recovery plan considerably more optimistic than the backup.

Figure 3 summarizes which compromises change availability, policy enforcement, or asset authority. It also makes explicit that recovery infrastructure can become an alternative route to effective quorum control. A

backup ceremony that aggregates enough shares or can roll back every comparison checkpoint deserves the same scrutiny as the signing service.

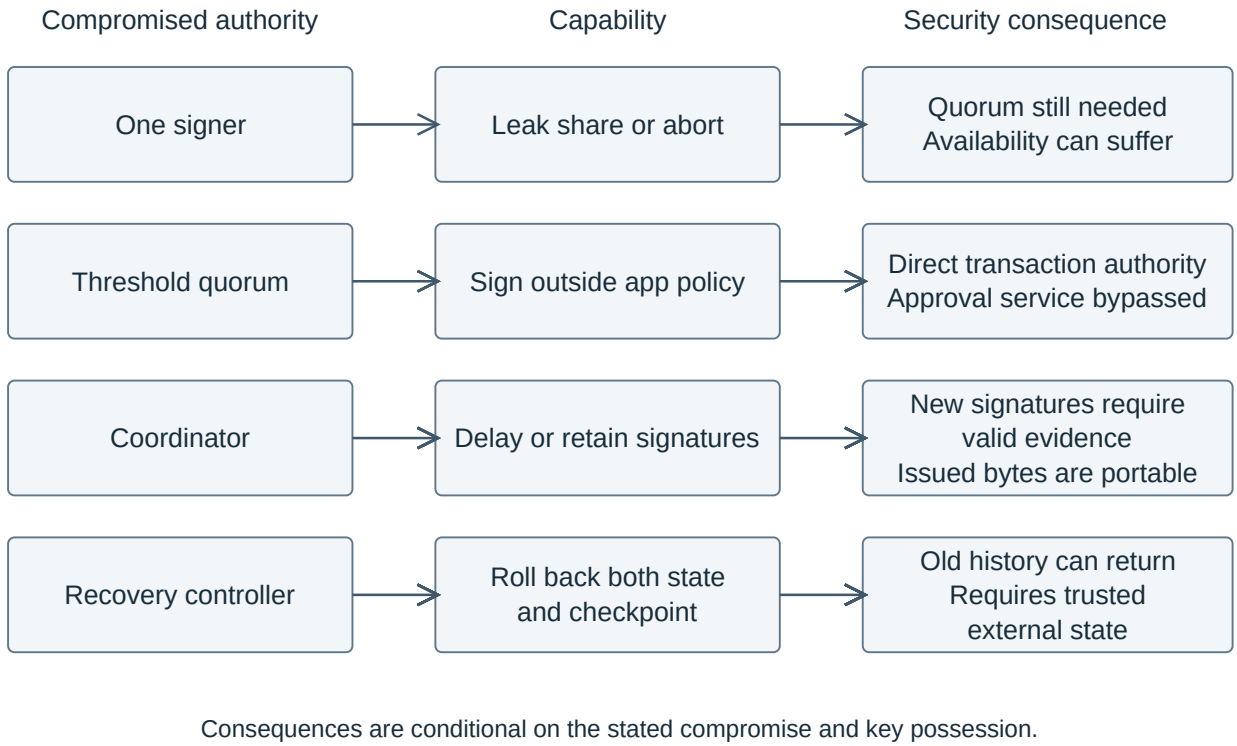


Figure 3. Compromise and recovery boundaries. Effects depend on the declared attacker capability, approval evidence, available signed bytes, and control of comparison checkpoints. A single compromised signer is not a compromised quorum.

9 Experimental Security Evaluation

The repeated repository unit suite completed 26 tests with zero failures and zero skips. The added research harness completed its assertions and a real local chain transfer. Table 2 classifies the added evidence. An experiment may produce an informative narrower result while leaving the requested full architecture sequence Not Executed. The paper does not count a model, a complete key fixture, or an imported validator as threshold integration.

Table 2. Added experiments and actual execution scope

Experiment	Observed outcome	Evidence boundary
A Spending history	1.200084 ETH modeled against 1 ETH budget; central model rejects second	Executed model + authorize; real MPC and PostgreSQL Not Executed
B Approval expiry	Validator rejects; direct Anvil receipt status 1; duplicate nonce rejected	Executed validator + ordinary key + chain; full worker/MPC Not Executed
C Recovery rollback	Stale/missing/changed checkpoint rejected; joint rollback accepted	Executed recovery functions; separate administrators/live replay Not Executed
D Availability	Retry transitions allowed; confirmed to signing rejected	Pure state checks only; native outage/fault cases Not Executed

9.1 Divergent Spending Histories

Hypothesis A is that a signing threshold alone does not establish an aggregate spending history. The model gives A, B, and C a 1 ETH lifetime budget. A is malicious; B and C are honest. Two real application authorization envelopes, with two distinct Ed25519 approvals each, authorize 0.6 ETH transactions at nonces 0 and 1. Each reserves 0.600042 ETH including the maximum fee. A hides its first participation before cooperating with C.

Both honest local model views accept their own 0.600042 ETH reservation, while the combined reservation reaches 1.200084 ETH. A central serial reservation model accepts the first and rejects the second. This executes a counterexample model and the actual authorize function. It does not execute the signer's HTTP handler, native MPC, or PostgreSQL locks. Source inspection shows the application's central wallet reservation check in src/custody.ts; preservation of that check under real concurrent database transactions remains untested in this added experiment. The result is a trust boundary demonstration, not an application exploit.

9.2 Expiry and Portable Transaction Authority

Hypothesis B is that expired application approval can prevent a fresh application submission decision while leaving an already signed ordinary Ethereum transaction valid. The harness creates a local type 2 transaction for 0.01 ETH, obtains real detached approvals, validates them through authorize, signs with an ordinary synthetic ECDSA key, and persists the public signed bytes. It waits beyond expiry before calling the unmodified validateBroadcastDecision function.

The validator rejected the decision with approval_expired. Direct submission to Anvil then produced receipt status 1 in block 1. A repeated submission returned nonce too low and the sender's account nonce remained 1. The transaction hash and timestamps are recorded in Appendix B. This demonstrates application validator rejection and actual chain acceptance. The ordinary key fixture deliberately isolates signature portability; obtaining the signature from real MPC and driving the database worker were Not Executed.

9.3 Recovery Rollback

Hypothesis C is that the checkpoint provides rollback protection only while its history remains independently trustworthy. The harness backs up a synthetic signer directory containing a reservation and a prepared session marker, advances its ledger and checkpoint, removes the storage directory, and attempts recovery. An older backup was rejected with stale_recovery_backup; a missing checkpoint failed with ENOENT; and a changed comparison digest rejected the backup.

When the declared privileged attacker restored both the old checkpoint and its matching encrypted backup, the repository recovery functions accepted them and preserved the old reservation and session marker. Application code was unchanged. This is the documented same host trust limitation, not a bypass under an uncompromised checkpoint assumption. Relocation into another directory also succeeded with the same identity and checkpoint, but both directories had the same OS administrator. Independent administration, live replay endpoint enforcement, and recovery from a genuinely interrupted signing or submission workflow were Not Executed.

9.4 Availability and Failure Handling

Hypothesis D is that lifecycle validation alone cannot establish distributed availability. Direct transition checks accepted signing to ready and signing to blocked, and rejected confirmed to signing. Anvil rejected a duplicate included transaction without advancing the nonce again. These checks concern transition permissions and chain nonce behavior. They do not measure the worker's retry limits, lease recovery, or signer reconnection behavior.

Real DKG with an absent participant, signing with one participant offline, coordinator interruption, a native signing timeout, participant disconnection, and interrupted native signing retries were Not Executed because the full native and database environment was unavailable. Historical repository results are retained as historical evidence only. A successful threshold signing claim requires the real integration path, with its source revision recorded.

9.5 Test Evidence Quality

The random input parser property test catches every Error and treats that as an acceptable outcome. Because its internal assertions are also Errors, that structure can absorb an assertion failure after an unexpected successful parse. Consequently, its configured 1,000 iterations should not be described as strong independent proof of the acceptance contract. The separate amount range property and input schema checks provide different coverage. This observation comes from inspecting the test body, not from observing an exploit or a failed test.

The new scripts assert the outcomes described above and preserve result JSON with a declared scope for every observation. Correcting the existing property test is proposed rather than silently changing the application's baseline tests. No latency or reliability estimate for custody operations is inferred from the short unit suite.

10 Quantum Threats to Ethereum Custody

Threshold ECDSA reduces dependence on a single holder of the signing secret under its classical assumptions. It leaves the public verification relation unchanged. A quantum attacker able to solve the relevant elliptic curve discrete logarithm problem can derive x from $Q = xG$ and sign directly. The attacker does not have to ask two participants, steal a recovery bundle, or negotiate with the custody policy engine. Shor's work establishes the algorithmic basis of the discrete logarithm threat on a sufficiently capable quantum computer [11].

This is an assumption change, not evidence of a current operational attack against the inspected wallet. A credible discussion must distinguish an algorithm from a machine capable of executing a cryptographically relevant instance with sufficient error correction and useful runtime. This paper supplies no hardware resource estimate, attack completion time, or forecast year. Procurement decisions should not depend on an invented countdown.

Exposure is also more specific than saying that every address publishes a public key. A conventional Ethereum address is derived from a hash of the public key; an outgoing signed transaction permits recovery of the ECDSA public key from its signature and message. A wallet's public key may also be exposed through service metadata or other systems. A future discrete logarithm attack against a known public key differs from a preimage search against an address whose key is unknown. Avoiding outgoing transactions is not a complete migration strategy, and this distinction does not justify treating all unspent accounts as equally exposed [7], [9].

The prototype's detached Ed25519 approvals create a second dependency. They also rely on an elliptic curve discrete logarithm assumption. Replacing the final transaction signature while retaining quantum vulnerable policy or recovery authority can leave a bypass. The migration inventory must cover approver identities, administrator signatures, software update roots, transport authentication, and any authority that can replace the wallet's verifier. A chain is not quantum resistant merely because its most visible signature changed.

The exposure distinction changes what an institution should inventory. It should record whether an account's public key is derivable from outgoing transactions, disclosed through wallet metadata, or shared with counterparties. Signature verification also depends on administrator and approver public keys that may

already be public to participants. Hashing a blockchain address does not hide every authority in the surrounding custody system.

The relevant institutional timescale is the migration lead time. Discovering dependencies, implementing verifiers, auditing contracts, rehearsing recovery, changing approval devices, and moving assets all take coordination. That work can begin without claiming that a cryptographically relevant quantum computer has arrived. The uncomfortable part of quantum planning is that uncertainty about the attack date does not make the integration work disappear.

11 Post Quantum Cryptographic Alternatives

ML-DSA offers lattice based signatures using polynomial arithmetic and hash expansion, with security tied to the standardized lattice assumptions. SLH-DSA builds signatures from hash based components and authentication paths. Both change the public verification problem attacked by Shor; neither supplies custody governance by itself [12], [13]. ML-KEM, specified by FIPS 203, establishes shared secrets and is relevant to communication protection, not direct replacement of a transaction signature [20].

Table 3 gives exact encoded sizes for selected parameter sets. ML-DSA-44, -65, and -87 target NIST security categories 2, 3, and 5; SLH-DSA-SHA2-128s targets category 1. The selection is not a claim of equal security. The hybrid size is the sum of a compressed secp256k1 key and compact 64 byte signature with ML-DSA-65, excluding any framing, algorithm identifiers, Ethereum recovery byte, or account metadata.

Table 3. Encoded sizes and security categories for selected schemes [7], [12], [13]

Scheme	Security basis or category	Public key bytes	Signature bytes
ECDSA secp256k1	Elliptic curve discrete logarithm	33 compressed	64 compact
ML-DSA-44	Lattice assumptions; category 2	1,312	2,420
ML-DSA-65	Lattice assumptions; category 3	1,952	3,309
ML-DSA-87	Lattice assumptions; category 5	2,592	4,627
SLH-DSA-SHA2-128s	Hash based; category 1	32	7,856
ECDSA AND ML-DSA-65	Both verifiers required; composition dependent	1,985 sum	3,373 sum

11.1 Local Verification Microbenchmark

The benchmark generates one key and one signature for each scheme over the same 32 byte message. It checks acceptance and altered message rejection, performs five warmups, then measures seven batches of thirty valid verifications. Table 4 reports the median and range of batch mean milliseconds. This measures warm JavaScript CPU verification on one Apple M3 Max with Node 22.20.0. It does not measure signing cost, independent input populations, native optimized code, HSM execution, gas, or account abstraction admission.

Table 4. Local CPU verification time in milliseconds per operation

Scheme	Median of batch means	Min to max batch mean
ECDSA secp256k1 compact	1.166	1.155 to 1.213
ML-DSA-44	1.094	1.078 to 1.133
ML-DSA-65	1.770	1.741 to 1.783
ML-DSA-87	2.766	2.726 to 2.820
SLH-DSA-SHA2-128s	1.125	1.120 to 1.229
Hybrid AND ECDSA + ML-DSA-65	2.954	2.878 to 2.969

A small CPU time does not imply inexpensive EVM verification. The EVM exposes different operations and existing precompiles; calldata volume, memory, hashing, polynomial arithmetic, canonical decoding, and worst case malformed inputs can dominate account validation. SLH-DSA’s large signatures can matter even when a particular CPU verifier is quick. This study implements no Solidity post quantum verifier and reports no gas benchmark.

11.2 Institutional Suitability and Composition

Table 5. Verification and migration considerations

Approach	Verification and EVM challenge	Custody suitability	Migration complexity
ML-DSA	Polynomial arithmetic, hash expansion, strict decoding; no gas result	Candidate for mandatory contract authorization; implementation review needed	New verifier, devices, key lifecycle and recovery
SLH-DSA	Many hash operations; large calldata; no gas result	Alternative assumptions; signature volume and operational cost matter	Large message transport and account verifier integration
Hybrid AND	Run both verifiers; bind the same context; close downgrade paths	Transition can retain protection if one scheme holds under composition assumptions	Dual key ceremonies and consistent retirement rules
Contract account / ERC-4337	Validation interface, bundler limits and executable verifier	Can enforce PQ proof at asset boundary; not itself a signature scheme	Move EOA assets; audit owner, modules, upgrades and guardians
Distributed PQ signing	Additional protocol rounds, proofs and abort behavior	Research direction; no implemented custody guarantee here	Protocol selection, integration, refresh and compatibility validation

An AND policy can require classical and post quantum proofs for the same operation. A simple concatenation benchmark demonstrates only the runtime cost of checking both signatures, not a formally analyzed hybrid signature construction. Bindel and Hale distinguish hybridization goals and caution that parallel signing does not automatically achieve all desired properties; their cited note is explicitly work in

progress [24]. The account must bind algorithm choices, keys, chain, nonce, operation bytes, and policy epoch, and must reject downgrade routes.

Distributed post quantum signing adds a separate implementation question. TOPCOAT and functionally interchangeable lattice threshold work demonstrate credible research directions [22], [23]. Their assumptions, parameter sets, output formats, and abort behavior need review before integration. Standardized ML-DSA signatures should not be assumed to emerge from any protocol called threshold Dilithium. No distributed post quantum implementation, key refresh protocol, or signature compatibility test is executed here.

The benchmark implementation itself is a research dependency. Its project documentation states that it has not received an independent audit [25]. Passing these local checks is not FIPS validation or evidence of side channel resistance. An institution should select a maintained, independently reviewed implementation and exercise failure behavior through the actual account and signing devices.

12 Proposed Migration Architecture

An internal post quantum approval in front of an ordinary ECDSA EOA improves an honest service’s approval process. It does not stop an adversary who can derive the EOA private key and submit directly. The decisive question is what the chain requires before assets move. A useful migration must make the new authorization mandatory at that boundary and close older routes that still permit spending or verifier replacement.

ERC-4337 provides an account abstraction framework in which a smart contract account determines how a UserOperation is validated. Its signature field is interpreted by the account implementation rather than fixed to one application signature format, and its replay rules bind the chain and EntryPoint context [15]. This creates a potential route for post quantum verification without requiring every Ethereum EOA transaction to change format. It does not establish that a particular verifier is economical, audited, or supported by every bundler.

Figure 4 presents a proposed transition rather than a deployed Quorum Custody feature. Phase 1 inventories classical dependencies and asset exposure. Phase 2 rehearses hybrid account authorization with synthetic assets and reviewed governance. Phase 3 moves assets and operational authority to accounts with a versioned verifier interface, explicit downgrade rules, and recovery paths that preserve the new authorization requirements.

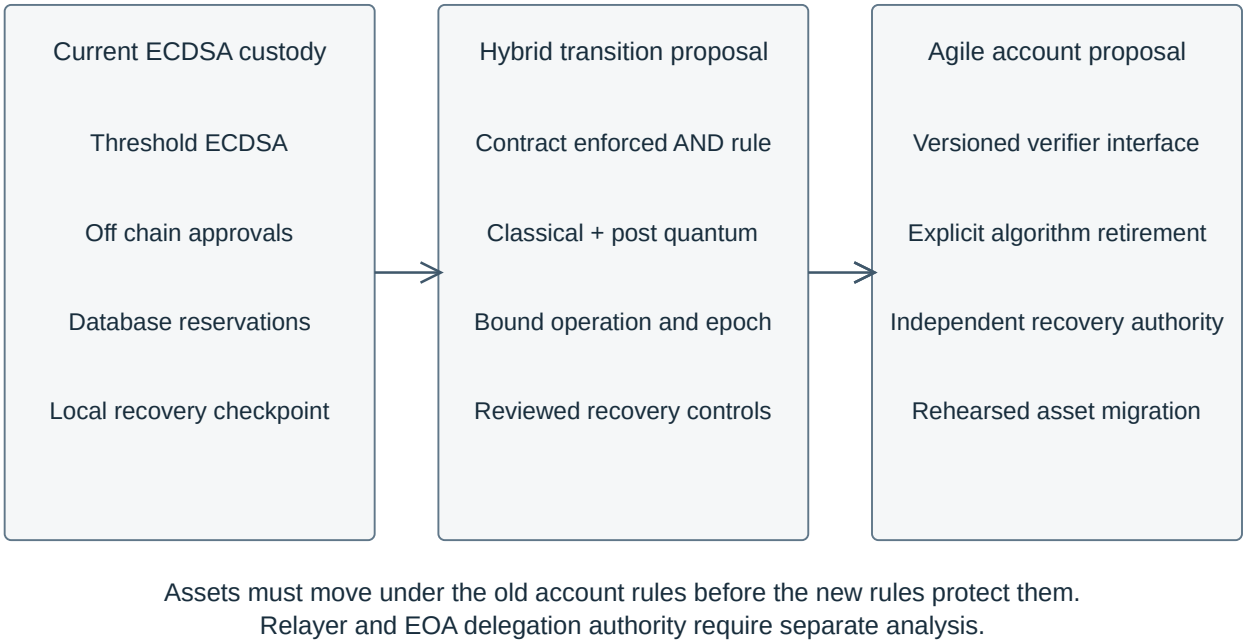


Figure 4. Proposed post quantum custody migration. The transition requires asset movement and protection of upgrade and recovery authority. These phases are proposed and have not been deployed or tested as a contract account.

12.1 Inventory and Account Preparation

The authority inventory should cover EOA keys, threshold shares, approver and policy keys, transport certificates, update roots, guardians, account modules, upgrade administrators, and recovery services. Classify which authority can spend directly, issue a signature, alter a verifier, or restore effective quorum control. Link those authorities to asset balances and account public key exposure. Prioritization should follow asset authority and migration difficulty rather than key count alone.

Deploy and independently review a candidate contract account before moving assets. Its operation format should identify the signature algorithm and verifier version, bind chain and account context, and reject unknown combinations. Governance should specify who can rotate keys, when a verifier upgrade becomes effective, how old operations expire, and what emergency control can halt. An emergency key with unrestricted spending or verifier replacement recreates a shortcut around the primary policy.

12.2 Hybrid Enforcement and Asset Movement

A proposed institutional account could require both the existing authorization and a post quantum signature during a transition period. The two signatures must bind the same operation, account, chain, nonce, and policy epoch. An AND rule can preserve protection when one scheme fails, subject to correct composition and verification. An OR rule that leaves classical ECDSA sufficient retains the classical bypass. These are proposed design choices, not implemented Quorum Custody features.

Every upgrade and recovery path deserves the same scrutiny. A quantum vulnerable administrator key that can install a new verifier defeats a post quantum spending rule. So can a recovery guardian, an unrestricted module, or a legacy owner function that remains sufficient to move funds. An account that enforces the new scheme only on its preferred user interface has achieved an interface migration. The attacker will use the other function.

A relayer’s ordinary transaction key can still be classical without necessarily controlling contract assets, provided the contract validates the required authorization independently and the relayer has no privileged spending or upgrade role. Compromising that relayer may deny service or consume its own gas funds. This is distinct from a classical owner key that the contract accepts as sufficient authority. ERC-4337 changes the validation interface; it does not guarantee this separation [15].

Existing assets must leave the old EOA through a valid authorization under the old rules. Rehearse the movement, reconcile balances and allowances, and retire every old account path that retains authority over the migrated assets. An existing EOA cannot be declared migrated merely because its interface forwards to a post quantum approval service. EIP-7702 delegation needs separate evaluation because the original ECDSA root remains relevant [26].

12.3 Recovery Rotation and Cryptographic Agility

Rehearse recovery with independent administrators before holding valuable assets in the new account. Key rotation must advance an authorization epoch, invalidate obsolete operations as specified by the contract, and preserve nonce and spending history. A retained external checkpoint needs an explicit owner, integrity mechanism, rollback model, and loss procedure. Recovery must not silently fall back to a classical key that the threat model already treats as forgeable.

Long term agility requires versioned parsers, bounded signature lengths, explicit algorithm retirement, and a tested migration path for the next verifier. Account upgradeability adds flexibility and an additional authority that can destroy the intended guarantees. A fixed account avoids that particular upgrade route but still needs a way to migrate assets if a scheme or verifier fails. Neither choice eliminates governance work.

Contract authorization does not make the entire surrounding network quantum resistant. Consensus signatures, client security, provider behavior, and chain governance retain their own assumptions. The narrower goal is to stop a classical account key compromise from being sufficient for asset movement inside the selected account design. That is an achievable design objective to investigate, provided it is stated with its boundary intact.

13 Discussion and Institutional Implications

The first improvement for the case study is operational independence. Separate signer storage, wrapping authority, administration, release permissions, and recovery custody across domains that one routine account cannot jointly replace. Approval devices need comparable protection because an attacker who acquires enough approval keys can ask honest signers to perform a policy permitted theft. Hardware protection should be selected against a stated host compromise model, with evidence showing what remains outside the hardware boundary.

The next priority is consistent authorization state. If coordinator and database compromise are in scope, define a reservation mechanism with explicit fault assumptions, policy epochs, revocation propagation, and rollback protection. Rehearse the failure cases before adding richer asset types. Supporting tokens and arbitrary Ethereum Virtual Machine (EVM) calls expands the semantic review problem: a small calldata change can authorize a durable allowance or invoke an unexpected contract path. The current ETH restriction avoids those complexities rather than resolving them.

The experiments illustrate three different meanings of policy. Exact approval signatures authenticate a transaction intent. A reservation history decides whether several authenticated intents fit a budget. Chain validation decides whether portable signed bytes can move assets. A control can work correctly at one of these boundaries while another boundary permits an outcome the institution dislikes. The correct response is to locate the missing enforcement, rather than describe the entire system as either secure or broken.

The observed rollback acceptance has the same structure. An external comparison checkpoint prevents selected stale restoration errors only if the attacker cannot roll it back with the state. Independence is therefore a property of administration and storage authority. It should be tested through real permissions and recovery exercises, not inferred from a filesystem path.

Institutional adoption also requires client ownership records, asset segregation, reconciliation, incident response, retention, and operational accountability beyond this prototype's signing flow. This paper makes no legal compliance or custody authorization finding. A source based engineering inventory can help identify work; it cannot supply missing organizational controls.

14 Limitations and Future Research

The case study is one local prototype at one revision. Its three signers do not demonstrate independent production trust domains. Additional tests use synthetic state, and the availability assertions exercise a pure transition function. The expiry test uses real approval validation and Anvil but an ordinary complete ECDSA fixture key. It does not establish that the full native MPC worker rejects or reconciles the same case correctly. Table 2 and Appendix B retain those boundaries explicitly.

Experiment A does not execute PostgreSQL reservations or a malicious native signer. Experiment C does not test separately administered hosts, a live replay endpoint after restore, or a real partially completed protocol. Experiment D's native failure scenarios are Not Executed. Full stack execution requires a running database environment, the pinned patched OpenSSL and cb-mpc build, and a provisioned isolated profile. The existing historical integration evidence is not counted as a new run.

The verification microbenchmark uses a single key and message per algorithm, a fixed algorithm order, and a warm JavaScript process. Its batch means are descriptive observations, not confidence intervals or production service estimates. The compared schemes have different security categories and implementation costs. No EVM verifier, gas profile, malformed input timing campaign, native optimized implementation, hardware assessment, or threshold post quantum protocol was benchmarked.

Further work should execute the current real stack, repair the parser property test, test reservation consistency under explicit coordinator and database faults, inject crashes at persistence boundaries, and rehearse total domain loss with independent recovery authority. A migration prototype should evaluate verifier correctness, bundler admission, upgrade control, guardian failure, and asset reconciliation before claiming operational readiness.

The reported signature portability and checkpoint trust limitations were already described by the repository and are not presented as previously unknown vulnerabilities. The test assertion handling observation is an evidence quality issue, not a demonstrated asset theft. If subsequent testing reveals a new exploitable weakness, preserve a minimal reproduction and coordinate privately with the maintainer before public disclosure of enabling details. No external disclosure was sent during this investigation.

15 Conclusion

The case study's strongest lesson is that custody security is cumulative. Distributed signing can constrain key misuse. Exact approval bindings can constrain a hostile coordinator. Persistent state can constrain retries and stale recovery. Asset boundary validation can preserve rules after a signature leaves the institution. Each control has a specific job and a specific failure boundary. The engineering task is to make those boundaries agree before entrusting them with assets that cannot be recalled.

The evidence supports a conditional answer to the research question: custody guarantees depend on the surrounding authorities as well as the signature protocol. Quantum migration must preserve those boundaries when spending, upgrade, and recovery keys change.

References

Sources accessed 21 October 2026. Repository references identify the original and experimental revision.

[1] demis1997, ‘Quorum Custody source code,’ GitHub, rev. 691e253ccdc2756afe215a23384eb72e2d4fde3c, especially src/domain.ts, src/encoding.ts, src/signer.ts, src/custody.ts, src/worker.ts, src/broadcast-decision.ts, native/, and tests/. [Online]. Available: github.com/demis1997/quorum-custody, pinned source. Accessed: Oct. 21, 2026.

[2] demis1997, ‘Threat model and security limitations,’ Quorum Custody, inspected revision. [Online]. Available: github.com/demis1997/quorum-custody, pinned source. Accessed: Oct. 21, 2026.

[3] demis1997, ‘Authorization at broadcast’ and ‘First roadmap slice execution report,’ Quorum Custody, docs/engineering/AUTHORIZATION_BOUNDARY.md and VERIFICATION.md, inspected revision. [Online]. Available: github.com/demis1997/quorum-custody, pinned source. Accessed: Oct. 21, 2026.

[4] demis1997, ‘Architecture,’ Quorum Custody, inspected revision; cb-mpc upstream pin 0b716706f998633912ee2be9b9bf8909aed66088. [Online]. Available: github.com/demis1997/quorum-custody, pinned source. Accessed: Oct. 21, 2026.

[5] demis1997, ‘Verification record,’ Quorum Custody, inspected revision. Historical execution evidence is revision specific. [Online]. Available: github.com/demis1997/quorum-custody, pinned source. Accessed: Oct. 21, 2026.

[6] National Institute of Standards and Technology, Security Requirements for Cryptographic Modules, FIPS PUB 140-3, Mar. 2019, doi: 10.6028/NIST.FIPS.140-3. [Online]. Available: https://csrc.nist.gov/pubs/fips/140-3/final. Accessed: Oct. 21, 2026.

[7] Standards for Efficient Cryptography Group, SEC 1 Elliptic Curve Cryptography, ver. 2.0, May 21, 2009, sec. 4.1. [Online]. Available: https://www.secg.org/sec1-v2.pdf. Accessed: Oct. 21, 2026.

[8] T. Pornin, Deterministic Usage of the Digital Signature Algorithm and Elliptic Curve Digital Signature Algorithm, RFC 6979, Aug. 2013, doi: 10.17487/RFC6979. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6979. Accessed: Oct. 21, 2026.

[9] V. Buterin et al., ‘EIP-1559 Fee market change for ETH 1.0 chain,’ Ethereum Improvement Proposals, Apr. 2019. [Online]. Available: https://eips.ethereum.org/EIPS/eip-1559. Accessed: Oct. 21, 2026.

[10] demis1997, ‘Development wallet recovery,’ Quorum Custody, inspected revision. [Online]. Available: github.com/demis1997/quorum-custody, pinned source. Accessed: Oct. 21, 2026.

[11] P. W. Shor, ‘Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer,’ SIAM J. Comput., vol. 26, no. 5, pp. 1484–1509, 1997, doi: 10.1137/S0097539795293172. [Online]. Available: https://arxiv.org/abs/quant-ph/9508027. Accessed: Oct. 21, 2026.

[12] National Institute of Standards and Technology, Module-Lattice-Based Digital Signature Standard, FIPS PUB 204, Aug. 2024, doi: 10.6028/NIST.FIPS.204. Consult the publication’s current errata when implementing. [Online]. Available: https://csrc.nist.gov/pubs/fips/204/final. Accessed: Oct. 21, 2026.

[13] National Institute of Standards and Technology, Stateless Hash-Based Digital Signature Standard, FIPS PUB 205, Aug. 2024, doi: 10.6028/NIST.FIPS.205. [Online]. Available: https://csrc.nist.gov/pubs/fips/205/final. Accessed: Oct. 21, 2026.

[14] National Institute of Standards and Technology, ‘Post-Quantum Cryptography FIPS Approved,’ Aug. 13, 2024. [Online]. Available: https://csrc.nist.gov/News/2024/postquantum-cryptography-fips-approved/. Accessed: Oct. 21, 2026.

[15] V. Buterin et al., ‘ERC-4337 Account Abstraction Using Alt Mempool,’ Ethereum Improvement Proposals, Sept. 2021, living specification. [Online]. Available: https://eips.ethereum.org/EIPS/eip-4337. Accessed: Oct. 21, 2026.

[16] R. Gennaro and S. Goldfeder, “Fast Multiparty Threshold ECDSA with Fast Trustless Setup,” Cryptology ePrint Archive, Paper 2019/114, revised Dec. 2021; underlying work presented at ACM CCS 2018. [Online]. Available: https://eprint.iacr.org/2019/114. Accessed: Oct. 21, 2026.

[17] M. Castro and B. Liskov, “Practical Byzantine Fault Tolerance,” in Proc. 3rd Symp. Operating Systems Design and Implementation, 1999, pp. 173–186. [Online]. Available: https://www.usenix.org/conference/osdi-99/presentation/practical-byzantine-fault-tolerance. Accessed: Oct. 21, 2026.

[18] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of Distributed Consensus with One Faulty Process,” J. ACM, vol. 32, no. 2, pp. 374–382, 1985, doi: 10.1145/3149.214121. Author manuscript available from MIT. [Online]. Available: https://groups.csail.mit.edu/tds/papers/Lynch/MIT-LCS-TR-282.pdf. Accessed: Oct. 21, 2026.

[19] E. Barker, Recommendation for Key Management Part 1 General, NIST SP 800-57 Part 1, Rev. 5, May 2020, doi: 10.6028/NIST.SP.800-57pt1r5. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final. Accessed: Oct. 21, 2026.

[20] National Institute of Standards and Technology, Module-Lattice-Based Key-Encapsulation Mechanism Standard, FIPS PUB 203, Aug. 2024, doi: 10.6028/NIST.FIPS.203. [Online]. Available: https://csrc.nist.gov/pubs/fips/203/final. Accessed: Oct. 21, 2026.

[21] D. Joseph et al., “Transitioning organizations to post-quantum cryptography,” Nature, vol. 605, pp. 237–243, May 2022, doi: 10.1038/s41586-022-04623-2. [Online]. Available: https://www.nature.com/articles/s41586-022-04623-2. Accessed: Oct. 21, 2026.

[22] N. Snetkov, J. Vakarjuk, and P. Laud, “TOPCOAT: towards practical two-party Crystals-Dilithium,” Discover Computing, vol. 27, art. 18, July 2024, doi: 10.1007/s10791-024-09449-2. [Online]. Available: https://link.springer.com/article/10.1007/s10791-024-09449-2. Accessed: Oct. 21, 2026.

[23] G. Tang, B. Pang, L. Chen, and Z. Zhang, “Efficient Lattice-Based Threshold Signatures with Functional Interchangeability,” Cryptology ePrint Archive, Paper 2024/1067, 2024. [Online]. Available: https://eprint.iacr.org/2024/1067. Accessed: Oct. 21, 2026.

[24] N. Bindel and B. Hale, “A Note on Hybrid Signature Schemes,” Cryptology ePrint Archive, Paper 2023/423, revised July 2023. Work in progress. [Online]. Available: https://eprint.iacr.org/2023/423. Accessed: Oct. 21, 2026.

[25] P. Miller, “noble-post-quantum,” software release 0.7.1, and “noble-curves,” software release 1.9.7, used for local JavaScript verification experiments. [Online]. Available: https://github.com/paulmillr/noble-post-quantum. Accessed: Oct. 21, 2026.

[26] V. Buterin, S. Wilson, A. Dietrichs, and lightclient, “EIP-7702 Set Code for EOAs,” Ethereum Improvement Proposals, May 2024, living specification. [Online]. Available: https://eips.ethereum.org/EIPS/eip-7702. Accessed: Oct. 21, 2026.

Appendix A Reproducibility

The original baseline and additional application experiments use revision 691e253ccdc2756afe215a23384eb72e2d4fde3c. No application source changes were made. The package preserves the baseline document and builder, updated research content and builder, source license, exact npm lockfiles, four SVG diagrams with compatibility PNGs, experiment scripts, and sanitized results.

Environment: macOS ARM64, Apple M3 Max, Darwin 25.5.0, Node 22.20.0, npm 10.9.3. Anvil 1.3.5-nightly, commit 9cd8a9511bc7206cb251721fa42420a76c668ae3. Every chain operation used synthetic local balances. Event logs use UTC; the publication date uses Asia/Nicosia.

From the research-package directory, install and run in this order. Node 22.20.0 and Anvil must be on PATH. The setup script retrieves the public source only when it is absent and refuses a different revision. Registry access is required for dependency installation. No real credentials are needed.

```
bash experiments/setup.sh
bash experiments/run.sh
```

The runner invokes node --import tsx --test for the repository’s test files through its installed loader, then executes security-experiments.mjs and verification-benchmark.mjs. The former starts one loopback Anvil process on port 18545 by default and terminates only that process. Set RESEARCH_PORT to an unused loopback port if necessary. It does not start Docker or modify another service.

The benchmark uses five warmups and seven batches of thirty valid verifications per scheme over one 32 byte message. Keys and signatures are freshly generated and never logged. Results preserve each batch mean, exact public sizes, runtime and library versions. The hybrid row is a logical AND of two verifiers, not a new composite signature construction.

For full stack work, follow the repository’s pinned native build and isolated local profile instructions, then execute npm run integration and npm run resilience. Those commands are provided for future work and were Not Executed here. They require PostgreSQL and the compiled cb-mpc adapter with patched OpenSSL. Do not interpret the research runner as a substitute for them.

Document regeneration uses build_paper.py with python-docx and the supplied figure assets. toc-pages.json contains automatically derived final page locations for the cached TOC field. The distribution PDF was generated and every page inspected. Reflow after editing requires updating the TOC field and reexporting the PDF.

Appendix B Experiment Protocols and Results

A Divergent ledger protocol

Expected outcome: valid individual approvals do not establish one budget history. Threat: A equivocates about local history, B and C retain honest partial views, and both intents already have sufficient approvals. Procedure: run the two authorize calls at nonces 0 and 1, apply the declared ledger model to A+B and A+C, then apply the central serial model. Observed: 600042000000000000 wei per reservation, 1200084000000000000 wei combined, versus a 1000000000000000000 wei limit. The central model rejects the second request. Relevant result: A record in security-experiments.json. Actual SQL reservations and native signing: Not Executed.

B Expiry protocol

Expected outcome: application decision rejects after expiry while unchanged signed bytes remain acceptable to Anvil. Threat: a holder retains previously authorized ordinary signed bytes. Procedure: create and approve a 0.01 ETH type 2 transaction, sign with a synthetic complete key, persist bytes, wait for real wall clock expiry, call validateBroadcastDecision, then submit directly. Observed: approval_expired; Anvil receipt status 1 in block 1; duplicate submission rejected with nonce too low; account nonce remains 1. Full worker and threshold signature generation: Not Executed.

C Rollback protocol

Expected outcome: an independently retained latest checkpoint rejects obsolete backup state, while joint rollback falls outside that assumption. Threat: compare a stale backup holder with a privileged actor able to write the checkpoint and use the backup password. Procedure: backup synthetic prepared session state, advance ledger/checkpoint, remove storage, attempt stale/missing/changed checkpoint restores, then restore both matching old state and checkpoint. Observed: stale_recovery_backup, ENOENT, and stale_recovery_backup respectively; joint rollback accepted. Relocation also accepted under the same administrator. Live replay, independent hosts, and actual interrupted MPC/RPC recovery: Not Executed.

D Availability protocol

Expected outcome: transition permissions admit selected retry states and forbid terminal state regression; no native availability conclusion follows. Threat: interrupted work and duplicate delivery. Procedure: call ready to signing, signing to ready, signing to blocked, signed to broadcast, and confirmed to signing; resubmit B’s included bytes. Observed: permitted transitions succeed, confirmed to signing throws invalid_transition, and the chain rejects the duplicate nonce. Real one signer outage, DKG outage, coordinator kill, timeout, participant disconnect, and native retry: Not Executed.

B public transaction identifier

0xffa529747ba093795fb1b9ff8ddbd727d9ccd5c46aa777bb36634d5d4247653d

UTC event times: authorized 2026-10-21T15:36:00.118Z; approval expiry 2026-10-21T15:36:04.118Z; application rejection 2026-10-21T15:36:04.220Z; submission/receipt record 2026-10-21T15:36:04.288Z. Policy version 1. The local chain has been stopped; this hash is not a public network explorer reference.

Complete commands, all event records, benchmark batch values, and the repeated 26 test TAP log are supplied in results/. The public signed transaction file contains only local synthetic transaction bytes. No share, approval private key, password, bearer credential, or real asset is included. RESEARCH_LIMITATIONS.md enumerates every unexecuted scope.